Fractional Differencing: How to Save Time Serie Memory

Using Newton's method to make financial timeseries stationary

ZAUR T
APR 15, 2026

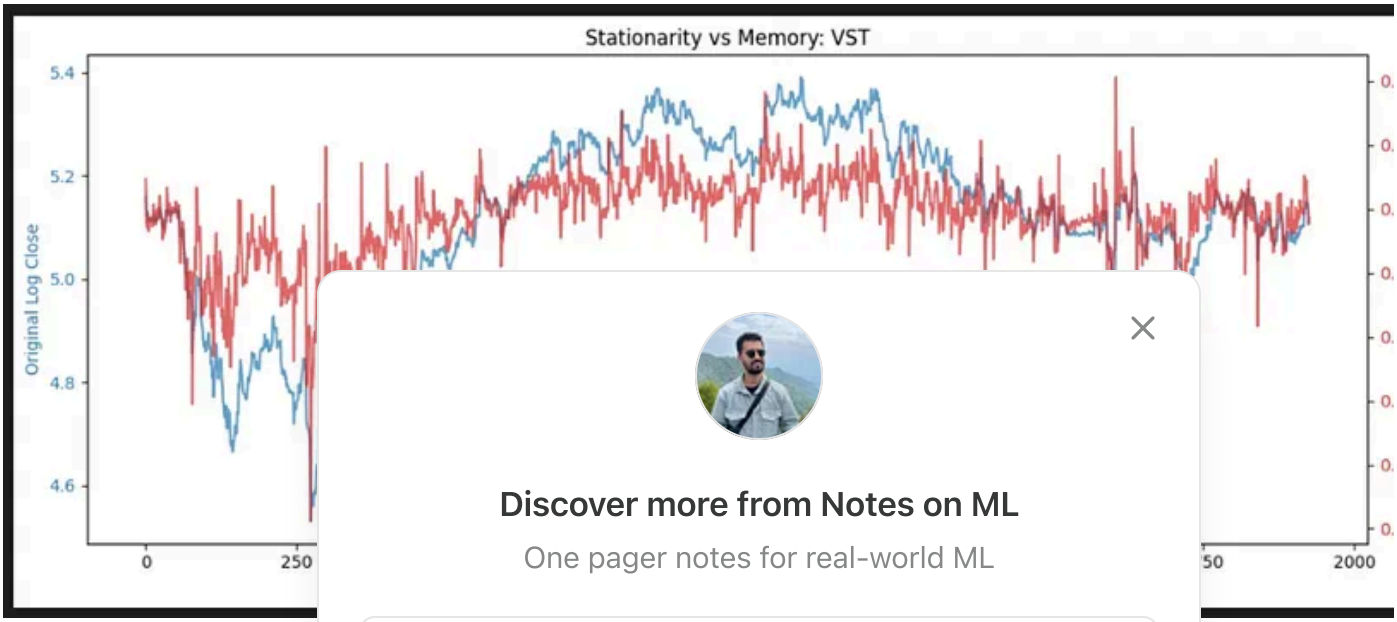
4 Sh

I was staring at the test results for my financial dataset; the correlation between my newly transformed dataset and the original stock prices was an incredible 0.98. Yet, statistical test immediately flagged the entire series as *non-stationary*.

Even worse, while trying to debug this, I realized I had accidentally made a fatal mistake. A quantitative investing sin that poisoned the dataset by leaking future pri into the past.

This is the story of how I implemented fractional differencing using concepts described by Marcos Lopez de Prado in his book, *Advances in Financial Machine Learning*. We're going to build the math of fractional differencing from the ground u translate it into high-performance Python code, and explore the silent statistical tra that will ruin your backtests if you aren't careful.

For those who want to jump directly into code, here's the repo.



Discover more from Notes on ML
One pager notes for real-world ML
Enter your email...
Subscribe
By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.
Already have an account? Sign in

Stationarity v

If you've ever fed fi n XGBoost, Ridge, or Neural N **must be stationary.** Station) of the serie don't change over t s entirely different dataset. V t. If you mak your training data stationary, using `.pct_change()` ,`.diff()` or whatever logic y use to calculate return, your model will fail to generalize. This is what generic mach learning tutorial teaches you. Use **standard differencing** and you'll be good.

For financial data it creates a problem. Because by doing this, we destroy the **memo**

Look at the price of NVIDIA, today's price is not a random. It's a chain reaction of historical events, built-up momentum, and institutional trades executed sequentiall over weeks. When you subtract yesterday's price from today's price, you wipe that history clean. You give the model amnesia. Even if you look back 3, 5, 10, n periods, you are only providing short-term memory.

Created with TradingView

If we don't difference, the data is non-stationary and our models break. If we do difference, the data becomes stationary but memoryless and useless for prediction.

Until we do fractional difference. In other words, we do difference on fractions inst of integers.

Don't confuse it with lags. When you apply a difference at a specific lag (let's call th lag k), you are building a bridge directly from Today to that exact day in the past. Yo completely ignore everything that happened in between

If you like what you're reading, share it with a friend. That will be a signal for me to do more of

these. Thanks!

Building the Math from Scratch

Imagine a mathematical operator, which we'll call **B** (the Backward Shift operator). only job of B is to serve as an index that reaches backward in time. `Lag` in other wo

If we have our current price X_t, then multiplying it by B gives us yesterday's price. Polars or Pandas, this is simply `df.shift(1)`)

$$B^1 X_t = X_{t-1}$$

Now, let's write out the standard differencing ("Returns") equation using our new operator

$$(1 - B)^1 X_t = X_t - BX_t$$

This is the same as `Current Price - Yesterday's Price`.

What if we difference it *twice* to capture more volatility? We raise the power to 2:

$$(1 - B)^2 X_t$$

If you remember high school algebra, you can expand a quadratic equation like (a-b into (a^2 - 2ab + b^2). Applying that to our operator, we get:

$$(1 - 2B + B^2) X_t$$

Drop the operator into the timeline, and the formula becomes (The current price, minus two times yesterday's price, plus the price from two days ago):

$$X_t - 2X_{t-1} + X_{t-2}$$

Notice a pattern?

If we start increasing the power number in our quadratic question like (a+b)^3, and then (a+b)^4 and so on, we'll be able to generalize the formula. The pattern from powers are exactly this sum which also works both ways.

$$\sum a^{n-i} \times b^{i}$$

This is the exact pattern of the [Binomial Theorem]() mapped through [Pascal's triangl]()

$$\begin{array}{ccccccccccccccc}
 & & & & & & & 1 & & & & & & & \\
 & & & & & & 1 & & 1 & & & & & & \\
 & & & & & 1 & & 2 & & 1 & & & & & \\
 & & & & 1 & & 3 & & 3 & & 1 & & & & \\
 & & & 1 & & 4 & & 6 & & 4 & & 1 & & & \\
 & & 1 & & 5 & & 10 & & 10 & & 5 & & 1 & & \\
 & 1 & & 6 & & 15 & & 20 & & 15 & & 6 & & 1 & \\
1 & & 7 & & 21 & & 35 & & 35 & & 21 & & 7 & & 1
\end{array}$$

The coefficients derived from expanding the polynomial act as specific **weights** assigned to past days. *For the reference this is exactly Binomial Distribution definition fr the probability class about random variables*

Subscribe so you don't miss the next issue.

Type your email... Subscribe

The Fractional Step: Escaping Integers

Now here things become interesting. What happens if we replace the integer expon (like 1 or 2) with a decimal? What if we differentiate the time series by d = 0.4?

$$(1 - B)^{0.4} X_t$$

Because 0.4 is a real decimal number, the binomial expansion never neatly closes. T
longer we go into the past the closer the weights are approaching zero, but never ze

The coefficients inside that infinite expansion turn into fractional weights for every
single past day in our dataset's history. The current day always receives a heavy wei
of 1.0. As we iteratively look further backward in time, the weights slowly decay,
asymptotically approaching zero, but they never actually hit zero.

We have just successfully created a formula for a **weighted average of all historical
memory**.

If we set d=1, we wipe out the memory completely (Returns). If we set d=0, we keep
original series (Raw Prices).

The issue now is to find an optimal decimal. Will get back to it later, but now there'
critical mathematical catch we have to solve before we can code this.

The Factorial Problem

To expand a binomial equation mathematically, you need to calculate factorials (n!).

If you ask Python to calculate the factorial of 5!, it easily computes `5 * 4 * 3 *
* 1`. But if you try to calculate the factorial of our decimal 0.4! your code will fail. T
formula attempts 0.4 - 1, generating a negative fraction, which crashes the calculati

Furthermore, if we iterate backward and multiply integers like `(d - 0) * (d -`
eventually our integer hits zero, which instantly multiplies everything by zero and
erases our entire sequence. This is a notorious trapdoor in Taylor Series expansions

The Solution: Newton's Generalization and the Morphing Ratio.

Rather than strictly calculating impossible factorials, we calculate a chain using the
product operator (\prod). Because the numerator and denominator in the combinati

formula share massive overlapping sequences, they cancel each other out.

Instead of computing every weight from scratch, we use a morphing transformatio ratio

$$\omega_k = -\omega_{k-1} \times \frac{d-k+1}{k}$$

Where k is the number of days we are looking backward.
We take the *previous* day's weight (e.g., w_{k-1}), and simply multiply it by this dynamically adjusting ratio to get the next historical weight.

To stop the loop from going back to the 1970s and freezing our RAM, we introduce mathematical threshold: `if weight < 0.00001: break`. Because weights belo that threshold have statistically zero impact on today's price.

Math to Python

I implemented this algorithm using the `polars` library for maximum high-performance execution speed (src/data/transform.py). In the file you'll also find features we built by using pandas and numpy described in this article

Here is how the logic of FFD works under the hood:

1. We compute a static Python list of weights once using the transformation loop defined above. We start with `[1.0]` for the current price, and append the morphing weights.
2. We map those weights dynamically across the Polars Dataframe using `.shift(i)`.
3. Because our data contains multiple tickers, we compute each lag using Polars' window function `.over("ticker")`. This is crucial: it prevents the shifted

closing price of Apple from accidentally bleeding into the lagged sequence of Microsoft.

The calculations will look something like that:

`(1.0 × close_today) + (−0.4 × close_yesterday) + (−0.12 × close_2_days_ago) + (−0.06 × close_3_days_ago) ...`

```python

def get_weights_ffd(d: float, threshold: float, max_lags: int = 1000) → list[floa
    w = [1.0]
    k = 1
    while True:
        w_k = -w[-1] * (d - k + 1) / k
        if abs(w_k) < threshold or k ≥ max_lags:
            break
        w.append(w_k)
        k += 1
    return w

def frac_diff_polars(df: pl.DataFrame, col_name: str, d: float, threshold: float =

    weights = get_weights_ffd(d, threshold)
    expr = weights[0] * pl.col(col_name)

    for i, weight in enumerate(weights[1:], start=1):
        expr = expr + weight * pl.col(col_name).shift(i).over("ticker")

    return df.with_columns(expr.alias(f"{col_name}_frac_diff"))


def find_min_ffd(df_ticker: pl.DataFrame, col_name: str = 'close', max_d: float =
    """
    Find the minimum d that makes the series stationary.

    min_d: floor for d — skips d=0 (raw series) which can pass ADF by luck
           on finite windows without being truly stationary. Default 0.1.
    significance: ADF p-value threshold.
    """
    d_values = [d for d in np.linspace(0, max_d, 11) if d ≥ min_d]

    for d in d_values:
        df_diff = frac_diff_polars(df_ticker, col_name, d, threshold)
        diff_col_name = f"{col_name}_frac_diff"
```

```
        series_np = df_diff.get_column(diff_col_name).drop_nulls().to_numpy()

        if len(series_np) < 50:
            continue

        adf_result = adfuller(series_np, maxlag=1, regression='c', autolag='AIC')
        p_val = adf_result[1]
        if p_val < significance:
            return d

    return max_d

def find_global_d(df: pl.DataFrame, col_name: str = 'log_close', reference_ticker:
    if reference_ticker is not None:
        df_ticker = df.filter(pl.col("ticker") == reference_ticker)
        return find_min_ffd(df_ticker, col_name, max_d, threshold, min_d, signific

    tickers = df.get_column("ticker").unique().to_list()
    optimal_ds = []

    for ticker in tickers:
        df_ticker = df.filter(pl.col("ticker") == ticker)
        d = find_min_ffd(df_ticker, col_name, max_d, threshold, min_d, significanc
        optimal_ds.append(d)

    if not optimal_ds:
        return max_d

    optimal_ds.sort()
    idx = int(len(optimal_ds) * coverage_threshold)
    if idx ≥ len(optimal_ds):
        idx = len(optimal_ds) - 1

    return optimal_ds[idx]
```

Deep Dive into ADF

Remember the disaster I mentioned in the introduction? After generating these nev fractionally differenced columns, I ran them through an **Augmented Dickey-Fuller (ADF) test.**

The ADF test is the gold standard for checking if a dataset is stationary or if it contains a “unit root” (meaning it wanders unpredictably). The more negative the A

statistic, the stronger the evidence that the series is safe for Machine Learning mod

My series had an amazing 0.98 correlation with the raw prices. It perfectly preserve the memory. But the ADF test failed completely. The algorithm insisted my modifie data was non-stationary. Why?

Let's look under the hood of ADF. The test relies on an **Autoregressive Model** (**AR**) estimate relationships.

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{p} \delta_i \Delta y_{t-i} + \epsilon_t$$

That looks scary, but here is the breakdown: It tries to predict today's return Δyt by looking at the intercept α, a time trend βt, yesterday's price gamma (or phi) y_{t-1}), sum of all historical lagged effects up to p days, and finally, true random noise/error t). That's almost everything you need to know for now about ADF. But if you think y want to go deeper, this video doing great job explaining it.

So the problem that I faced with was that I accidentally hardcoded the evaluation la (`autolag = 1`). Meaning, my ADF test only looked backward by exactly 1 day.

Financial assets have intense serial correlation called momentum. Think about this large institutions don't deploy $50 million all at once; they split their execution into thousands of chunks over a week to hide their trades inside normal trading volume. Because of this, positive momentum normally bleeds from Monday into Tuesday.

By forcing the ADF test to only look at *one* day of history, the test saw this institutional momentum, confused it for wild non-stationary drift, and completely failed the test.

The fix: I replaced the hardcoded lag with the **Akaike Information Criterion** (**AIC**) The AIC dynamically figures out the exact number of lags necessary to account for institutional momentum without overfitting the noise. Once the model could prope

“see” the momentum, the error term (\epsilon_t) became true random noise, and the stationarity test passed.

The Lookahead Bias

But the statistical trap wasn’t the worst mistake I made.

Because the optimal stationarity is different for every stock’s volatility profile, I initially wrote a dynamic script to calculate an individualized optimal d fractional value for every single ticker from 2018 to 2026.

This is a **catastrophic mathematical sin**, known as Lookahead Bias. (If you want to learn more, I highly recommend reading [Seven Sins of Quantitative Investing]).

Why this is an issue: If I scan the entire history of a stock up to 2026 to optimize my fractional weights, and then my model begins backtesting trades in 2023, the model using a memory weight optimized by volatility from events in 2025 that haven’t happened yet. I was leaking the future into the past.

The Global Fix:
You cannot dynamically optimize data transformations across validation or test sets. To fix this, I introduced strict temporal isolation: `training_cutoff_date`.

I engineered a new `find_global_d` function. It scans the universe of assets exclusively in the training data split. Instead of individualizing the fraction, it calculates a universal d parameter that successfully pushes the **80th percentile** of all assets across the stationarity threshold. Once that single decimal parameter (e.g., d=0.4) is found in the safe training past, it is locked in perfectly and blindly applied to the testing future via `apply_features()`. No data leaks.

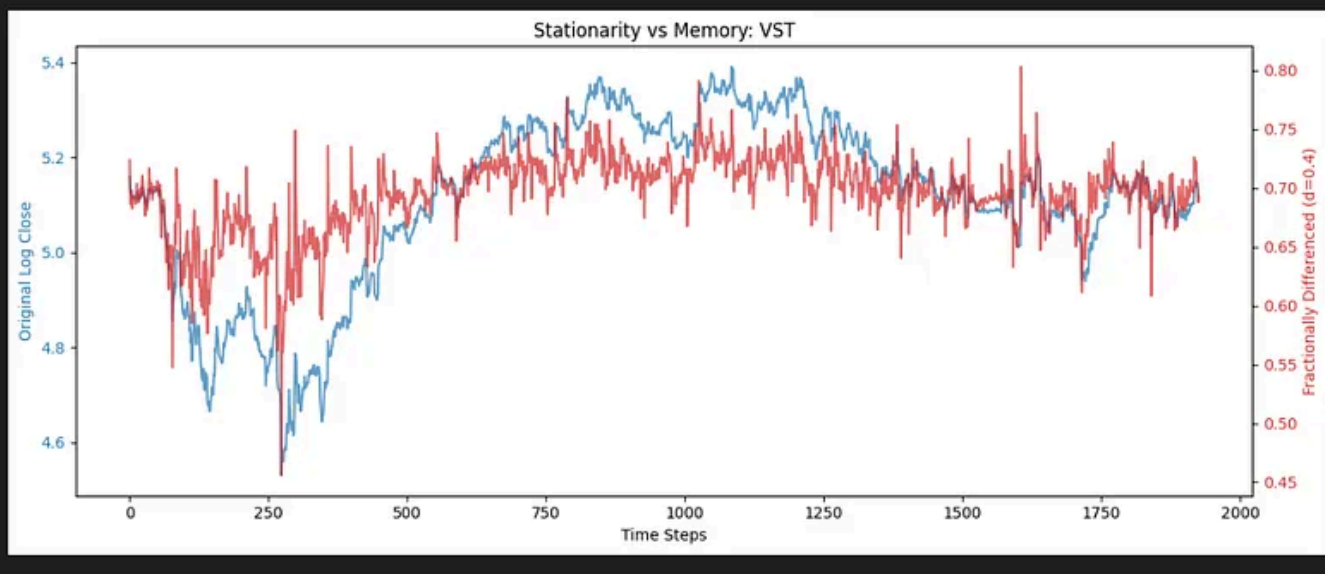
The Payoff: 85% Memory Retention

By applying this optimized global d=0.4 correctly without lookahead bias, we crosse the stationarity threshold ($p < 0.05$) across the dataset.

The resulting time series was completely stationary, mathematically safe for downstream machine learning applications. Yet, because we didn't force the intege 1, the data successfully retained **85.47%** of its original market memory and correlati

```
--- ADF Test Results for VST ---
Original 'log_close'     p-value: 0.5788 (Stationary: False)
FFD 'log_close' (d=0.4)  p-value: 0.0105 (Stationary: True)

--- Memory Retention ---
Correlation between original and FFD: 0.8509
```



We did not have to choose between keeping the memory and keeping the model.

The next time you are preparing a financial dataset, or any dataset that tracks evolv human behavior over time, pause before you blindly call `.diff()` or `.pct_change()`. Take those raw prices, apply fractional differencing to the decim and watch how much sharper your predictions become when your model finally remembers its past.

Subscribe to Notes on ML

By Zaur T · Launched 2 years ago

One pager notes for real-world ML

Type your email... Subscribe

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.

4 Likes

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Zaur T · Privacy · Terms · Collection notice
Substack is the home for great culture